GROUP 3

Machine Learning-Based Network Intrusion Detection System (NIDS) with Real-Time Alerting

PROJECT MANAGER:

DELOS REYES, MARY ROSE TAGLE

MACHINE LEARNING:

MARTINEZ, TRISHA MAE VIZCARRA

YAMBAO, JODIE AGBULOS

BACKEND:

ESPIRITU, KENETH RAMOS

ESTRADA, ARRIANE RUEDA

UI:

DOLIENTE, ARIES LAWRENCE ESTOQUE

SECURITY RESEARCH:

PADILLA, CLARISSE CEPE

RINGOR, LEENETH ANNE TRINIDAD

REPORT WRITING:

SANTOS, JOHN CARLO DE DIOS

SIMEON, ARVIN JAYSON MANANGHAYA

ABSTRACT

Network security is struggling against new "zero-day" cyberattacks because older rule-based tools only catch known threats and constantly trigger false alarms causing "alert fatigue" for security analysts. To solve this, our group built a Machine Learning-Based Network Intrusion Detection System (NIDS) that acts as an automated "digital security guard". We trained an artificial intelligence system using a dual-model approach on the NSL-KDD dataset. A Random Forest model was trained to catch known attacks, achieving a 99.71% F1-score with only a 0.19% false alarm rate. On the other hand, an Isolation Forest model memorizes what normal traffic looks like so it can instantly flag new and unseen attacks. We connected these models to a live, easy-to-read web dashboard that monitors traffic, instantly changes the threat level to "CRITICAL" during an attack, and categorizes the threats into 9 distinct types using the MITRE ATT&CK and STRIDE frameworks. Finally, we safely tested our system in an isolated virtual lab by launching a live port scan, proving the dashboard can catch hackers in real-time.

INTRODUCTION

Networks are constantly under attack by hackers trying to break in or shut systems down. For a long time, security teams have used rule-based tools like Snort and Suricata to protect their networks, but these older tools have a major flaw. They already have a strict rule, making them unable to detect new and unseen attacks. Worse, they constantly trigger false alarms that cause security teams to suffer from "alert fatigue". Alert fatigue is a state of exhaustion where analysts might accidentally ignore a real cyberattack because they are overwhelmed by fake warnings.

Our motivation for this project is to fix the problem of alert fatigue and improve network defense by applying artificial intelligence. Instead of relying on outdated rules, we wanted to build a system that dynamically learns what normal network behavior looks like and automatically flags suspicious anomalies without relying on known attack signatures.

To achieve this, our team built a Machine Learning-Based Network Intrusion Detection System (NIDS). Our system uses two AI models working together: a Random Forest to precisely catch known threats and an Isolation Forest to flag unusual, zero-day attacks. Then we connected these models to a live Streamlit web dashboard. Whenever malicious traffic is detected, the dashboard instantly alerts the user and categorizes the attack into one of 9 distinct threat types such as a "Smurf" DDoS attack or an "Nmap" port scan. It also explains the attacker's goals using professional security frameworks like MITRE ATT&CK and STRIDE. By building this system, we aim to provide security teams with a tool that not only detects threats accurately but also reduces the exhaustion caused by false alarms.

LITERATURE REVIEW

The Evolution of Detection

For decades, network security relied on a method known as signature-based or rule-based intrusion detection. Tools like Snort and Suricata operate under this approach by maintaining a large database of known attack patterns, called signatures. When network traffic passes through the system, it is compared against these stored signatures. If a match is found, an alert is triggered. This method is straightforward and effective against well-documented threats because the rules are written by security experts who have already studied and categorized specific attack behaviors (Garcia-Teodoro et al., 2009).

However, signature-based systems come with a major limitation: they can only detect what they already know. Any attack that does not match an existing rule in the database will pass through undetected. This is especially concerning in today's threat landscape, where attackers constantly modify their methods to avoid detection. New and unknown threats, often referred to as zero-day attacks, are completely invisible to traditional rule-based systems. Additionally, maintaining these signature databases requires continuous manual updates, which creates a gap between when a new threat is discovered and when protection against it is deployed (Liao et al., 2013).

This is where machine learning-based anomaly detection offers a meaningful advantage. Instead of relying on predefined rules, ML-based NIDS systems learn what normal network behavior looks like and then flag anything that significantly deviates from that baseline. Because the detection is based on behavior rather than known patterns, these systems are capable of identifying previously unseen attacks. Research has shown that ML-based approaches consistently outperform traditional rule-based systems when dealing with novel or evolving threats, making them a more adaptive solution for modern network security environments (Buczak & Guven, 2016).

The Mechanics of Machine Learning in NIDS

The NIDS developed in this project uses two machine learning models that serve different but complementary purposes: the Isolation Forest for unsupervised anomaly detection and the

Random Forest for supervised attack classification. Both models were trained on the NSL-KDD dataset, an improved benchmark for intrusion detection research that addresses the limitations of the original KDD Cup 99 dataset by removing duplicate records and providing a more balanced distribution of normal and attack traffic (Tavallaee et al., 2009).

The Isolation Forest is an unsupervised algorithm, meaning it does not require labeled training data to function. Instead of learning what attacks look like, it learns the structure of normal traffic. The algorithm works by randomly partitioning the data using decision trees. Data points that are easy to isolate, meaning they get separated from the rest of the data in very few splits are considered anomalies. The logic behind this is simple: abnormal data points are rare and structurally different from the majority, so they require fewer partitions to isolate (Liu et al., 2008).

In this project, the Isolation Forest was trained exclusively on benign traffic records. During testing, it achieved an F1-score of 85.66% and a false positive rate of 5.16%, which means it correctly flagged most anomalies while generating a manageable number of false alarms.

The Random Forest, on the other hand, is a supervised learning algorithm that was trained on labeled data containing both normal and attack records. It works by constructing a large number of decision trees during training, where each tree is built using a random subset of the data and features. When making a prediction, each tree votes on whether a connection is normal or an attack, and the majority vote becomes the final classification. This ensemble approach reduces the risk of overfitting that comes with using a single decision tree and generally produces more reliable results (Breiman, 2001). In this project, the Random Forest was trained on 88,181 samples across 12 selected features. It achieved an F1-score of 99.71% and a false positive rate of only 0.19%, significantly outperforming the Isolation Forest in terms of accuracy.

The decision to use both models together reflects the practical reality of network security. The Random Forest excels at identifying known attack types with high precision, while the Isolation Forest serves as a secondary layer of defense that can flag unusual activity even when it does not match anyknownattackcategory. This dual-model approach addresses one of the core weaknesses

of relying on a single detection method, giving the system both accuracy and adaptability (Abdulhammed et al., 2019).

Operational Cybersecurity Challenges (Alert Fatigue)

Alert fatigue is a common issue in modern Security Operations Centers (SOC), where security analysts are exposed to an overwhelming number of alerts generated by monitoring tools. A large portion of these alerts are false positives, which makes it harder to separate real threats from harmless system activities. Because of this overload, there is a risk that important security incidents may be ignored or delayed.

This problem often results in analyst exhaustion, lower efficiency, and a higher chance of missing actual cyberattacks. To reduce alert fatigue, organizations work on improving detection accuracy by lowering the False Positive Rate (FPR) and refining their security systems. This allows analysts to focus more on relevant and high-priority alerts, improving overall cybersecurity response (IBM Security, 2023).

MITRE ATT&CK Context & Synthesis

To better understand attacker behavior and improve threat analysis, the detected activities in the system are mapped using the MITRE ATT&CK framework, which categorizes common cyberattack techniques used in real-world incidents.

The use of nmap and portsweep tools corresponds to T1046 (Network Service Discovery). These tools are typically used by attackers to explore networks, identify active devices, open ports, and detect available services that could be targeted in future attacks.

The smurf attack is classified under T1498 (Network Denial of Service). It works by overwhelming a target system with excessive traffic, causing service disruption and making the system unavailable to legitimate users.

Rootkits are associated with T1014 (Rootkit). This technique allows attackers to conceal malicious actions within a system while maintaining long-term and hidden access without being easily detected.

Buffer overflow attacks fall under T1068 (Exploitation for Privilege Escalation). In this method, attackers take advantage of software vulnerabilities to gain higher-level access and potentially control system functions.

Password guessing attacks are categorized under T1110 (Brute Force). This involves systematically trying multiple password combinations until unauthorized access is achieved (MITRE ATT&CK, 2024).

The Neptune attack, which is a type of SYN Flood, is associated with T1498.001 (Network Denial of Service: Direct Network Flood) and may also be related to T1499.001 (Endpoint Denial of Service: OS Exhaustion Flood). In this attack, a large number of incomplete connection requests are sent to a target system, consuming its resources and eventually preventing legitimate users from establishing connections.

The ipsweep attack, also known as a Ping Sweep, is mapped to T1018 (Remote System Discovery). This technique is used to identify active devices within a network by sending ping requests to multiple IP addresses, allowing attackers to determine which hosts are reachable and potentially vulnerable.

The teardrop attack, which involves sending malformed or fragmented packets, is associated with T1499 (Endpoint Denial of Service). This attack exploits weaknesses in how systems handle packet reassembly, causing system crashes or instability that can disrupt normal operations (MITRE ATT&CK, 2024).

THREAT MODEL

MITRE ATT&CK			
Attack Type	**MITRE ID**	**TECHNIQUE NAME**	**Description**
Nmap/Port Sweep	T1046	Network Service Discovery	This attack involves scanning the network to identify open ports and active services, which attackers use to discover potential entry points.
Rootkit	T1014	Rootkit	This technique allows attackers to hide malicious processes or files, enabling them to maintain long-term access without being detected.
Buffer Overflow	T1068	Exploitation for Privilege Escalation	Attackers exploit software vulnerabilities to execute arbitrary code and gain elevated system privileges.
Smurf Attack	T1498.002	Reflection Amplification	A type of Distributed Denial of Service (DDoS) attack that overwhelms a network by flooding it with amplified traffic.

Guess Password	T1110	Brute Force	Attackers attempt multiple password combinations to gain unauthorized access to user accounts.
Neptune (SYN Flood)	T1498.001	Network Denial of Service: Direct Network Flood	This attack floods the target system with SYN requests, exhausting server resources and preventing legitimate connections.
IP Sweep (Ping Sweep)	T1018	Remote System Discovery	Attackers scan multiple IP addresses to identify active hosts within a network.
Teardrop (Malformed Packets)	T1499	Endpoint Denial of Service	This attack sends malformed packets that can crash or destabilize the target system.

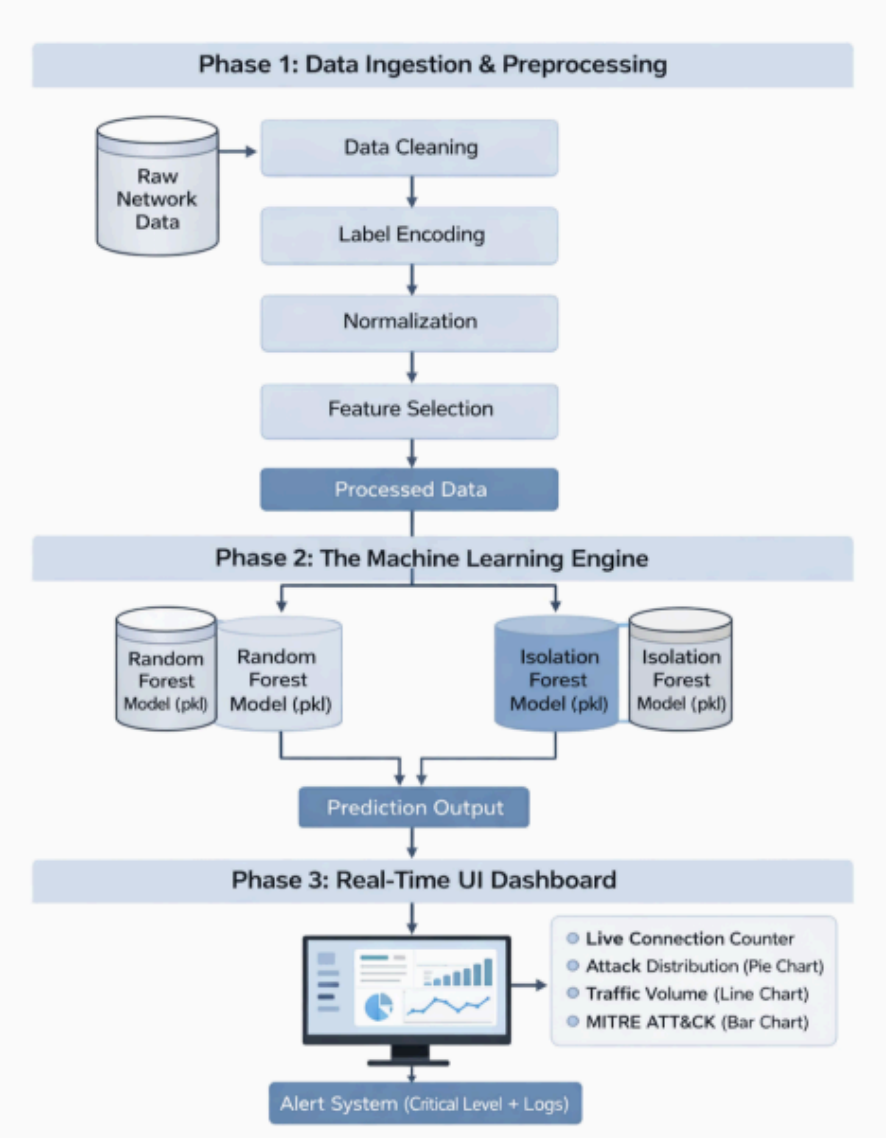
The system collects and analyzes network traffic, and when malicious activity is detected, it is mapped to corresponding MITRE ATT&CK techniques. Also, to common attacks such as port scanning and brute force, the system also detects more advanced threats like SYN flood (Neptune), ping sweep (IP sweep), and malformed packet attacks (Teardrop). These are mapped to their respective MITRE ATT&CK IDs to provide a standardized understanding of each threat. This method enhances threat visibility and allows security analysts to quickly identify the behavior and intent of attackers, improving incident response and decision-making.

STRIDE		
Stride Category	**Description**	**Example in the System**
Spoofing	Pretending to be a legitimate user or system entity	Mitigated by detecting abnormal login patterns such as repeated failed login attempts (brute force), which are flagged and logged in the dashboard.
Tampering	Unauthorized modification of data	Mitigated by analyzing network traffic patterns and detecting anomalies or unusual packet behavior that may indicate data manipulation.
Repudiation	Denial of performing an action without proof	Mitigated by the NIDS dashboard automatically logging exact timestamps, source IP addresses, and attack details in the alert table for every detected event.
Information Disclosure	Exposure of sensitive information	Mitigated by detecting reconnaissance attacks such as Nmap port scans and IP sweeps before attackers can identify vulnerable services.
Denial of Service	Disruption of system availability	Mitigated by instantly detecting and generating

		alerts for volumetric attacks such as Smurf and Neptune (SYN Flood), allowing immediate response.
Elevation of Privilege	Gaining higher-level permissions	Mitigated by identifying exploitation attempts such as buffer overflow attacks that aim to gain unauthorized administrative access.

The STRIDE model is used to categorize threats based on their potential impact on the system, while the NIDS provides corresponding mitigation through real-time detection and alerting. Instead of only identifying threats, the system actively monitors network behavior and logs suspicious activities, enabling early detection and response. For example, denial of service attacks such as smurf and SYN flood are mitigated through immediate alert generation, while reconnaissance activities like port scanning are detected before attackers can exploit system vulnerabilities. This approach ensures that each identified threat is not only classified but also addressed through the system's capabilities.

ARCHITECTURAL DIAGRAM



This system effectively processes and analyzes network traffic in real time using a three-phase pipeline. Before being examined by two machine learning models that identify both known and undiscovered risks, incoming data is first cleaned, encoded, and standardized to guarantee accuracy. After that, the findings are shown in a live Streamlit dashboard that alerts security analysts, categorizes the attack distribution, and maps the threats to the MITRE ATT&CK framework. The system can function as a "digital security guard" due to this pipeline, which continuously monitors connections and offers prompt threat detection and reaction.

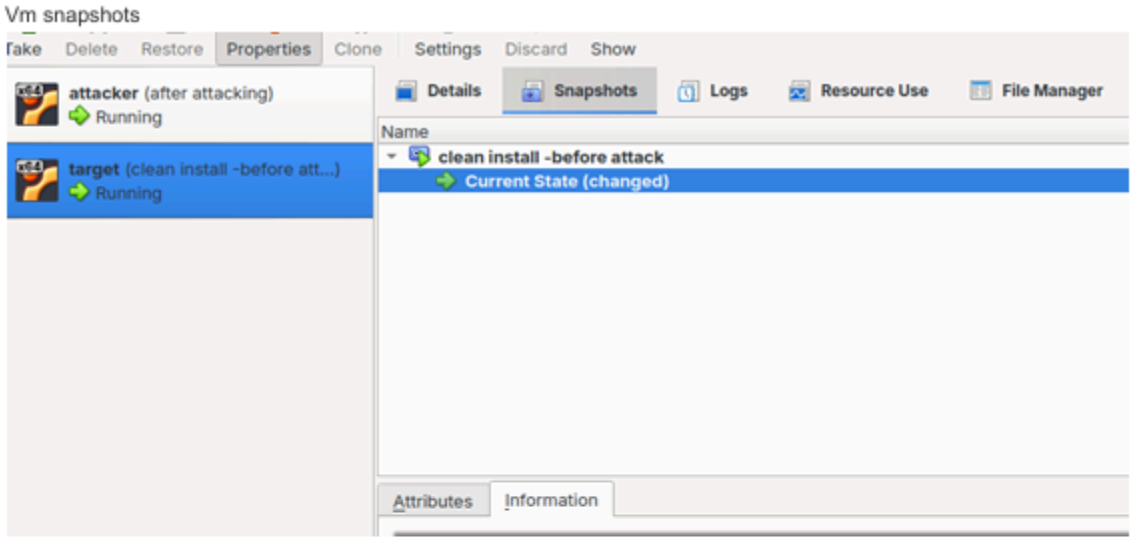
IMPLEMENTATION DETAILS

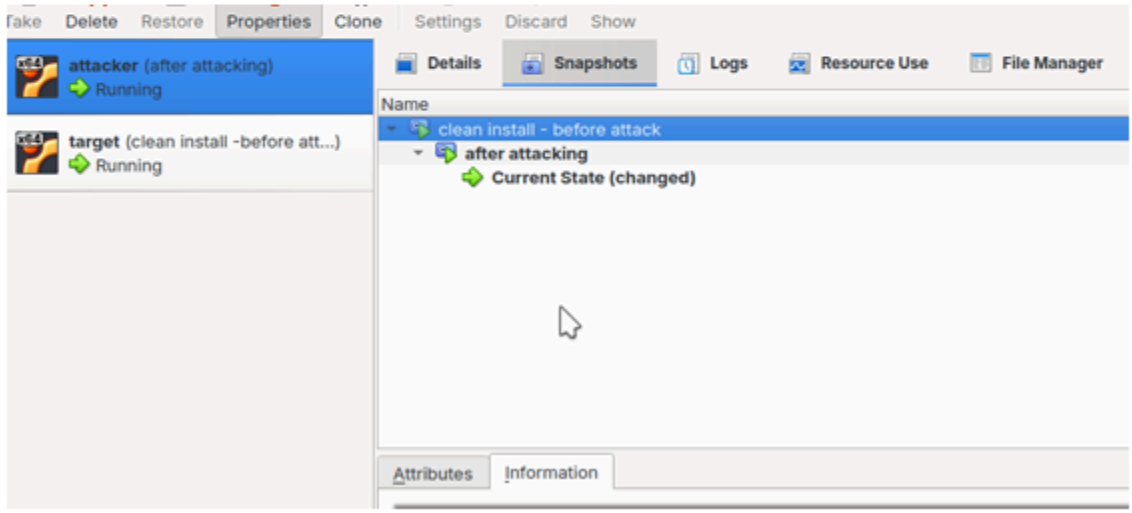
To build our Network Intrusion Detection System (NIDS), we first processed our raw network data using the NSL-KDD dataset. Using Python and the Pandas library, we loaded the data, ensured there were no missing values, and encoded the text labels into a binary format: normal traffic as "0" and all attack types as "1". We then normalized the numerical data using a StandardScaler and applied a feature selection technique to keep only the 12 most predictive data points out of the original 38. With the data prepared, we trained two machine learning models: a supervised Random Forest on the fully labeled data to catch known attacks, and an unsupervised Isolation Forest purely on the benign data to act as an anomaly detector. Both models and the scaling tools were saved using the joblib library, and we wrote a custom prediction script to identically process and score new, incoming network connections.

Next, we connected our machine learning backend to a live, interactive web application built with Streamlit. We designed this dashboard to display a live connection counter, a traffic volume line chart, and a threat level indicator. To provide professional context, we integrated the MITRE ATT&CK and STRIDE frameworks directly into the dashboard's backend logic. This ensured that any malicious activity processed by the ML models would be automatically logged and categorized into specific threat types (such as Smurf or Nmap attacks) within a filterable alert table and attack distribution pie chart.

Finally, to safely generate live traffic for our system to analyze, we built an isolated virtual lab using VirtualBox. We configured a "Host-only" network to ensure that no simulated attack traffic could escape to the real internet. Inside this lab, we deployed two virtual machines: an "attacker" machine and a "target" machine assigned to the 192.168.56.x subnet. We took VM snapshots (such as a "clean install - before attack" state) to create safe restore points, allowing us to instantly fix the machines if they broke during testing. We configured Wireshark on our host machine to continuously capture the live network packets flowing from the virtual machines and feed them to our Streamlit dashboard. To systematically test the system, we formulated an attack

plan using nmap from the attacker VM to execute various network scans against the target including a stealth SYN scan (-sS), a TCP connect scan (-sT), a service version detection scan (-sV), and an aggressive, high-intensity scan (-A -T4).





RESULTS AND EVALUATION

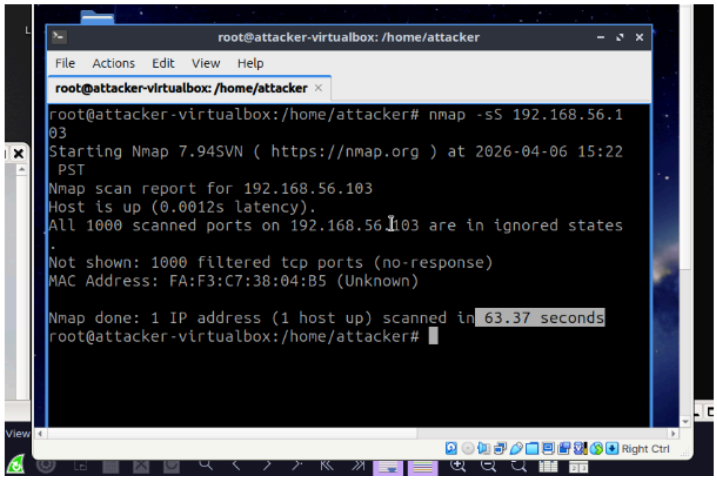
Our machine learning models delivered excellent results during testing. The Random Forest model proved to be highly accurate at catching known threats, achieving an impressive 99.71% F1-score while maintaining a very low false positive rate of just 0.19%. This high accuracy is important because it helps solve the problem of “alert fatigue,” where security analysts get too many false warnings. With fewer fake alerts, they can focus on real threats instead of wasting time checking harmless activity. At the same time, the Isolation Forest model learned what normal network behavior looks like without supervision. It produced a slightly higher false positive rate of 5.16%, meaning it sometimes flagged unusual but still normal traffic. However, this trade-off is expected in real-world use because it helps the system detect completely new or unknown “zero-day” attacks.

To evaluate the practical capabilities of the system, we ran live attacks inside our isolated virtual lab. We used Nmap to launch several port scans of varying intensities against our target machine, including a stealth SYN scan, a service detection scan, and an aggressive, high-intensity scan. Even though all 1,000 scanned ports on the target machine were completely closed, the attacks successfully generated the rapid network traffic required to trigger our models. The system analyzed the packets in Wireshark and responded almost immediately. The stealth SYN scan was completed in 63.37 seconds, the service detection scan took 58.60 seconds, and the aggressive scan took 80.46 seconds. Despite these longer scan durations, the live dashboard successfully monitored the traffic and instantly reacted to all of them. The system successfully changed the threat level indicator to a flashing 'CRITICAL' and logged 98 active threats in the filterable alert table as the attack was happening.

Furthermore, the system successfully categorized the malicious traffic into distinct attack types and automatically mapped them to professional security frameworks to help defenders understand the attacker's intent. For example, the live Nmap port scans were correctly identified and tagged with the MITRE ATT&CK ID T1046, which stands for Network Service Discovery. The system also accurately categorized other complex threats, mapping "smurf" attacks to Network Denial of Service (T1498) and "buffer overflow" attacks to Exploitation for Privilege

Escalation (T1068). Overall, these results prove that our system effectively uses artificial intelligence to instantly detect dangerous network behavior and clearly explains those threats to security teams so they can take immediate action.

nmap -sS 192.168.56.103 - 63.37 seconds



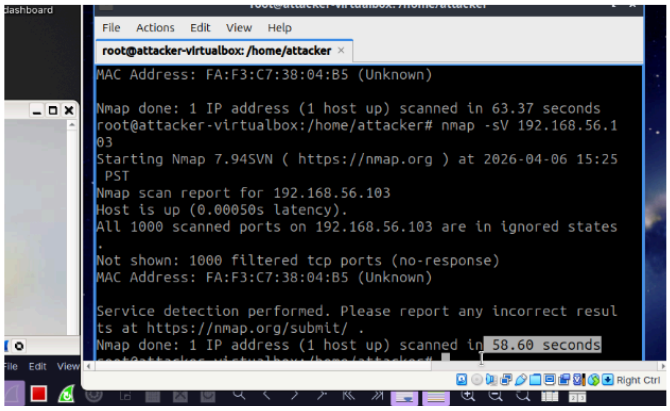
Suspicious Connections (Filterable Alert Table)

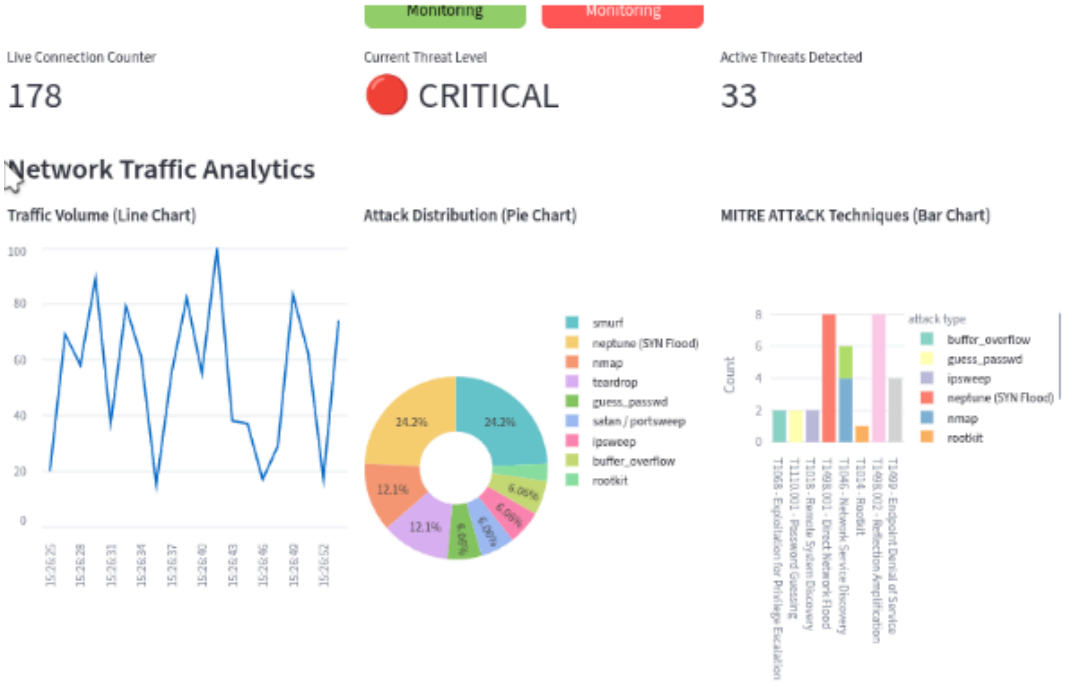
Filter by Attack Type

All

time	attack type	mitre id	mitre name
15:22:37	smurf	T1498.002	Reflection Amplification
15:22:41	smurf	T1498.002	Reflection Amplification
15:22:44	nmap	T1046	Network Service Discovery
15:22:55	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:22:57	teardrop	T1499	Endpoint Denial of Service
15:23:11	smurf	T1498.002	Reflection Amplification
15:23:21	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:23:28	guess_passwd	T1110.001	Password Guessing
15:23:30	smurf	T1498.002	Reflection Amplification
15:23:39	satan / portsweep	T1046	Network Service Discovery

nmap -sV 192.168.56.103 - 58.60 sec





Suspicious Connections (Filterable Alert Table)

Filter by Attack Type

All

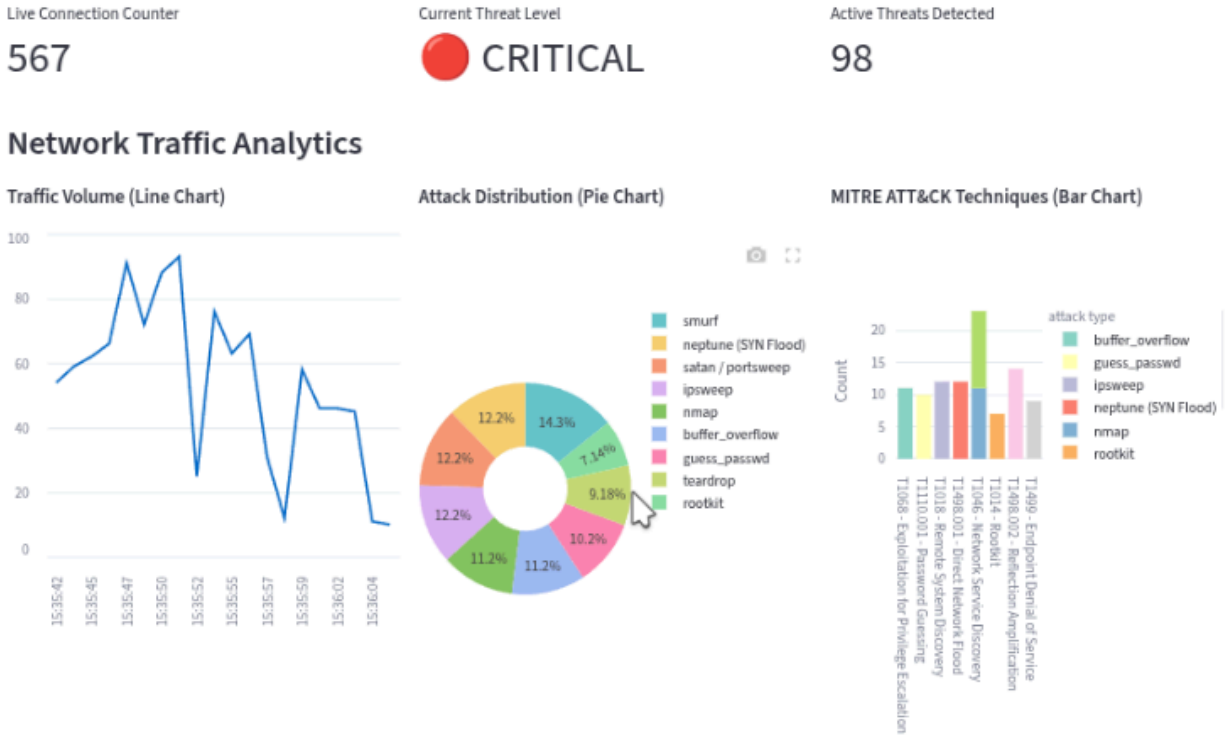
time	attack type	mitre id	mitre name
15:22:37	smurf	T1498.002	Reflection Amplification
15:22:41	smurf	T1498.002	Reflection Amplification
15:22:44	nmap	T1046	Network Service Discovery
15:22:55	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:22:57	teardrop	T1499	Endpoint Denial of Service
15:23:11	smurf	T1498.002	Reflection Amplification
15:23:21	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:23:28	guess_passwd	T1110.001	Password Guessing
15:23:30	smurf	T1498.002	Reflection Amplification
15:23:39	satan / portsweep	T1046	Network Service Discovery

nmap -A -T4 192.168.56.103- 80.46 sec

```
root@attacker-virtualbox:/home/attacker# nmap -A -T4 192.168.5
6.103
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-06 15:29
 PST
Nmap scan report for 192.168.56.103
Host is up (0.00039s latency).
All 1000 scanned ports on 192.168.56.103 are in ignored states
.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: FA:F3:C7:38:04:B5 (Unknown)
Too many fingerprints match this host to give specific OS deta
ils
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   0.39 ms 192.168.56.103

OS and Service detection performed. Please report any incorrec
t results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 80.46 seconds
root@attacker-virtualbox:/home/attacker#
```



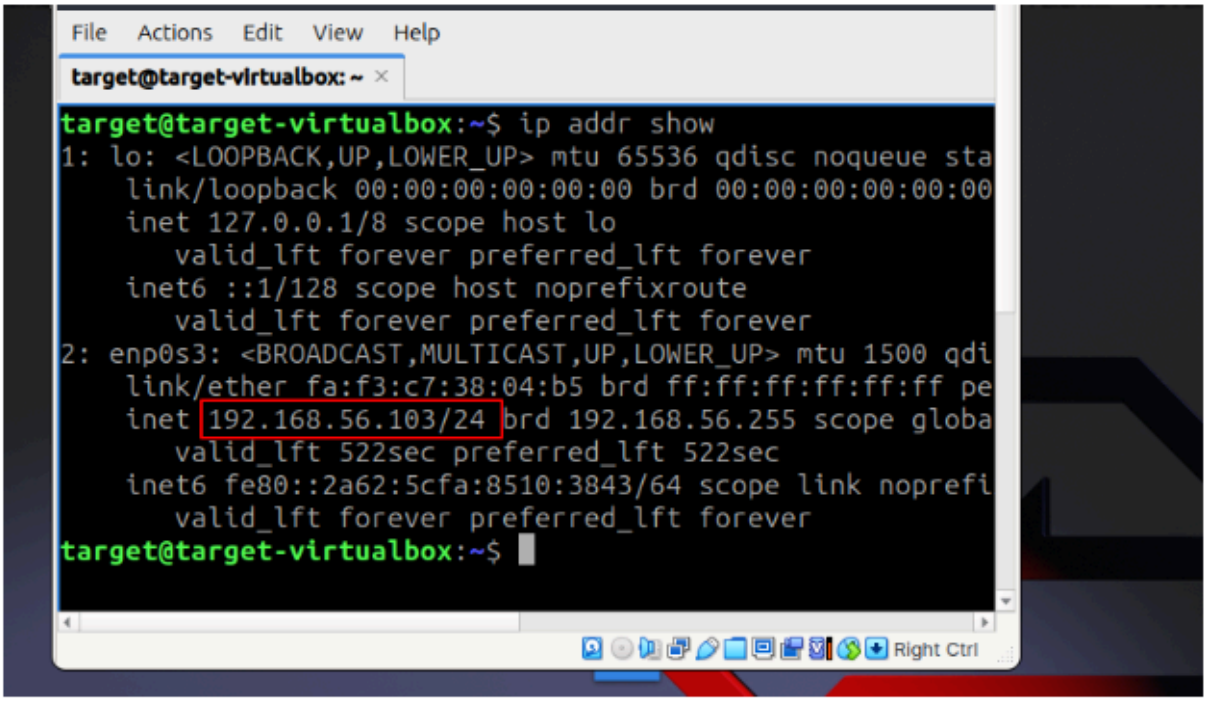
Suspicious Connections (Filterable Alert Table)

Filter by Attack Type

All

time	attack type	mitre id	mitre name
15:22:37	smurf	T1498.002	Reflection Amplification
15:22:41	smurf	T1498.002	Reflection Amplification
15:22:44	nmap	T1046	Network Service Discovery
15:22:55	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:22:57	teardrop	T1499	Endpoint Denial of Service
15:23:11	smurf	T1498.002	Reflection Amplification
15:23:21	neptune (SYN Flood)	T1498.001	Direct Network Flood
15:23:28	guess_passwd	T1110.001	Password Guessing
15:23:30	smurf	T1498.002	Reflection Amplification
15:23:39	satan / portsweep	T1046	Network Service Discovery

Target vb



LIMITATIONS AND FUTURE WORK

While our Network Intrusion Detection System (NIDS) performed very well, there are still a few limitations to how it currently operates. First, the system still struggles with false positives, meaning it sometimes flags normal network traffic as an attack just because the traffic pattern is uncommon. During our tests, the unsupervised anomaly detector had a 5.16% false positive rate. In a real-world setting, generating too many of these fake warnings can lead to "alert fatigue". Second, to ensure strict safety and ethical compliance, our system was only tested inside an isolated virtual lab environment using VirtualBox. We still do not know how the machine learning models would handle the massive, unpredictable data volume of a real corporate network.

For future work, our primary focus will be to tune the machine learning models to lower the false positive rate, which will directly reduce alert fatigue and make the system much more helpful for security analysts. We also want to expose the AI to more diverse datasets so it can learn to detect highly advanced attacks, rather than just the Nmap port scans we tested in our lab. Finally, we plan to upgrade the system from a passive monitor into an active defense tool. Instead of just flashing a "CRITICAL" warning on the dashboard, a future version could be programmed to automatically communicate with a network firewall to block the attacker's IP address the exact moment malicious traffic is detected.

ETHICS AND LEGALITY

When building and testing cybersecurity tools, following strict ethical and legal rules is the most important step. The most important rule is that you must never run hacking tools such as the Nmap port scanners we used on systems or networks that you do not own or do not have explicit written permission to test. Breaking this rule is not only unethical but it can also lead to serious legal consequences. We designed this project strictly for defensive research and educational purposes so that we could understand how cyberattacks work in order to stop them, not to cause harm.

To ensure we followed these strict rules, we performed all of our testing inside a completely isolated virtual lab. We used VirtualBox to create a "Host-only" network, which acted as a secure digital fence. This guaranteed that our simulated attack traffic was trapped inside our lab and could never escape to the real internet. Furthermore, we used virtual machine snapshots as "save points," which allowed us to instantly fix our computers if anything broke, ensuring no permanent damage was ever done.

Finally, because our system acts as a digital security guard that "listens" to network traffic using tools like Wireshark, it brings up important privacy considerations. Monitoring a network means collecting data packets, which can sometimes contain sensitive information. In a real-world environment, this type of monitoring must always be done responsibly, within strict legal boundaries, and with proper consent from the users on the network to ensure their privacy is protected.

CONCLUSION

Our team successfully built an AI-powered "digital security guard" to monitor network traffic and catch hackers in real time. By combining two different machine learning models, we addressed the major flaws of older, rule-based security tools that rely on outdated signatures. Our Random Forest model accurately caught known threats and significantly reduced annoying "false alarms" (which prevents analyst exhaustion known as "alert fatigue"), while our Isolation Forest model successfully learned to spot completely new, unseen "zero-day" attacks.

We successfully connected these AI models to an easy-to-read live dashboard that instantly flashes a "CRITICAL" warning when it spots danger. It also neatly categorizes the threats into 9 distinct types so security teams know exactly what the attacker is trying to do, mapping them directly to professional frameworks like MITRE ATT&CK and STRIDE. When we safely tested our system by simulating live Nmap cyberattacks inside a secure, isolated virtual lab, the dashboard caught the attacks almost instantly.

While the system is highly effective, it still occasionally flags normal traffic as suspicious, so our future goal is to fine-tune the AI to make even fewer mistakes and test it on more diverse corporate data. Ultimately, we plan to upgrade this project from a passive warning tool into an active defense shield that automatically blocks a hacker's IP address using a firewall the exact moment malicious traffic is detected.

References:

Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials, 18(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

Garcia-Teodoro, P., Diaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). Anomaly based network intrusion detection: Techniques, systems and challenges. Computers & Security, 28(1–2), 18–28. https://doi.org/10.1016/j.cose.2008.08.003

Liao, H. J., Lin, C. H. R., Lin, Y. C., & Tung, K. Y. (2013). Intrusion detection system: A comprehensive review. Journal of Network and Computer Applications, 36(1), 16–24. https://doi.org/10.1016/j.jnca.2012.09.004

Abdulhammed, R., Musafer, H., Alessa, A., Faezipour, M., & Abuzneid, A. (2019). Features dimensionality reduction approaches for machine learning-based network intrusion detection systems. Electronics, 8(3), 322. https://doi.org/10.3390/electronics8030322

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP99data set. In Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (pp. 1–6). IEEE. https://doi.org/10.1109/CISDA.2009.5356528

IBM. (2024). *IBM Security X-Force Threat Intelligence Index 2024*. Www.ibm.com. https://www.ibm.com/reports/threat-intelligence

Cisco. (2019). *Cisco - Global Home Page*. Cisco. https://www.cisco.com

Cybersecurity | Reports | CrowdStrike. (2024, September 25). Crowdstrike.com. https://www.crowdstrike.com/resources/reports

MITRE. (2024). *MITRE ATT&CK®*. Attack.mitre.org. https://attack.mitre.org

NIST. (2016, June 30). *Cybersecurity*. NIST. https://www.nist.gov/cybersecurity

Unit 42 - Latest Cyber Security Research | Palo Alto Networks. (n.d.). Unit42. https://unit42.paloaltonetworks.com

OWASP Foundation | Open Source Foundation for Application Security. (n.d.). Owasp.org. https://owasp.org